



Sprint 2

21.05.26



Stand-up (15 min)

- Was habe ich in dieser Woche gemacht?
- Was werde ich nächste Woche erreichen?
- Welche Probleme gibt es bzw. welche werden erwartet?



Deliverables für nächste Woche

1. Performance des Dynamics models

- 1-step & multi-step
- Loss zeigen
- Grafisch zeigen

2. Funktionsweise mit A2C/MCTS

- Zeigt Performance in CartPole
- Zeigt Performance in FrozenLake
- Vergleichen mit wahren Env. Dynamics (.step())

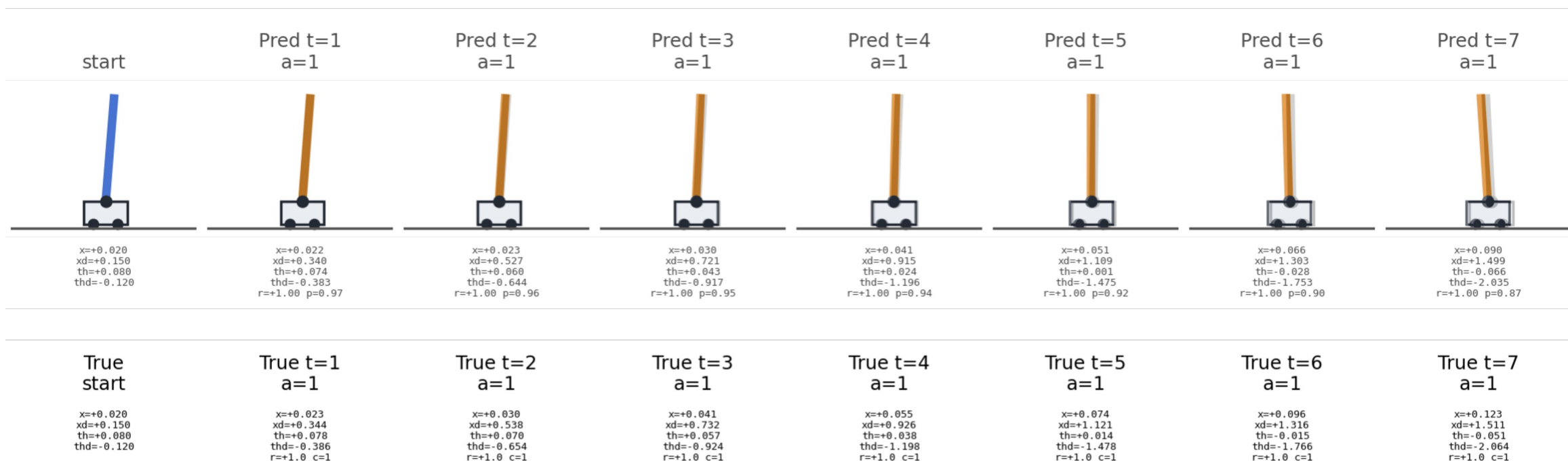


Vereinfachungen

- A2C
 - Time horizon für Training (z.B. nur 10 Schritte in die Zukunft)
 - Bootstrapping
 - Vorhersage von “done”
- MCTS
 - Rollouts mit wahren Dynamics (nicht vorhergesagt)



Learned dynamics model





FrozenLake

- Zufällige Start-position
- Bessere Reward-Informationen
- Leicht anderes Format/Architektur



FrozenLake

```
import gymnasium as gym
env = gym.make("FrozenLake-v1",
               map_name="4x4",
               is_slippery=False,
               reward_schedule=(1, -1, -0.01))
```



FrozenLake: NN Architektur

- Input: 16 + 4 (-> one-hot encoded version of state)
- Output: 16, 1
- Wir brauchen eine Wahrscheinlichkeitsverteilung über States
 - Softmax über NN outputs
 - Cross-entropy loss



```
loss = nn.MSELoss()  
output = loss(input, target)
```

```
loss = nn.CrossEntropyLoss()  
output = loss(input, target)
```



```
class TwoHeadNetwork(nn.Module):
    def __init__(self):
        super().__init__()

        self.shared_stack = nn.Sequential(
            nn.Linear(4, 64),
            nn.ReLU(),
            nn.Linear(64, 64),
            nn.ReLU()
        )

        self.head_1 = nn.Linear(64, 4)
        self.head_2 = nn.Linear(64, 1)

    def forward(self, x):
        x = self.flatten(x)
        features = self.shared_stack(x)

        out_1 = self.head_1(features)
        out_2 = self.head_2(features)

        return out_1, out_2
```



Erweiterung

$$f_{\theta}: s_t, a_t \rightarrow \hat{s}_{t+1}, \hat{r}_t, \hat{d}_t$$

